

3.2.2 Exemple pràctic amb Vite

Suposem que volem crear un projecte modern amb mòduls. Amb Vite, podem fer-ho molt fàcilment:

1. Obrim un terminal i executem:

```
1 npm create vite@latest projecte-vite
```

2. Triem el tipus de projecte (per exemple, “Vanilla” per JavaScript pur).

3. Entrem a la carpeta:

```
1 cd projecte-vite
```

4. Instal·lem les dependències:

```
1 npm install
```

5. I arranquem el servidor local:

```
1 npm run dev
```

Vite utilitza directament els **mòduls ES6** del navegador durant el desenvolupament, sense necessitat d’empaquetar res.

Només crea un **fitxer empaquetat i optimitzat** quan fem el desplegament final (npm run build).

3.2.3 Estructura d’un projecte Vite

Un cop s’ha creat un projecte amb Vite, la seva estructura d’arxius típica pot ser la següent:

```
1 /projecte-vite
2 |
3 |-- /node_modules          # Conté les dependències instal·
   lades
4 |-- /public                # Conté fitxers públics com el
   favicon o el HTML base
5 |   |-- index.html          # Fitxer HTML principal
6 |
7 |-- /src                   # Conté el codi font del projecte
8 |   |-- main.js             # Punt d’entrada de l’aplicació
   JavaScript
9 |   |-- style.css           # Arxiu CSS
10 |
11 |-- package.json           # Conté les dependències del
   projecte i configuració de npm
12 |-- vite.config.js         # Configuració de Vite
```

```
13 | -- /dist          # Conté la versió final del  
    | projecte, optimitzada per producció
```

Tant si utilitzem Vite com qualsevol altre empaquetador trobarem una estructura d'arxius força similar. Aquesta estructura pot variar, sobretot pot créixer si el projecte és gran, però a grans trets sempre veurem els projectes estructurats de forma similar.

Anem a analitzar cadascun dels arxius i carpetes principals:

- **/node_modules:** Aquesta carpeta conté totes les dependències del projecte que han estat instal·lades per **npm**. Aquesta carpeta és creada quan s'executa **npm install** i inclou totes les biblioteques i mòduls que el projecte necessita per funcionar.
- **/public:** Aquesta carpeta conté els arxius públics que el servidor utilitzarà. Generalment, inclou el fitxer **index.html**, que és el punt d'entrada de l'aplicació, així com altres arxius estàtics (com imatges o fonts).
- **/src:** Aquesta carpeta conté els fitxers de codi font de l'aplicació. És on es defineixen els mòduls JavaScript, les funcions d'operació (com les matemàtiques, en el cas de l'exemple), els arxius CSS per a l'estil, etc.
- **package.json:** Aquest fitxer és essencial per a la gestió de dependències de Node.js. Conté les següents seccions:
 - **dependencies:** Les biblioteques i mòduls que el projecte necessita per a funcionar.
 - **devDependencies:** Les eines que es necessiten només durant el desenvolupament (per exemple, Vite o altres eines de compilació).
 - **scripts:** S'hi defineixen comandes com **npm run build** o **npm start**, que són les que es poden executar des de la línia de comandes per realitzar tasques automàtiques com la construcció o el desenvolupament.
- **vite.config.js:** Aquest fitxer conté la configuració específica de Vite per a personalitzar l'entorn de desenvolupament o producció. Aquí es poden definir, per exemple, les rutes d'entrada, optimitzacions, etc.
- **/dist:** Aquesta carpeta és generada quan es fa **npm run build** i conté la versió optimitzada per a producció del projecte. És on es col·loquen els fitxers minificats, empaquetats i llestos per ser servits en un entorn de producció.

3.2.4 Per què fem servir un empaquetador?

Els empaquetadors no només uneixen fitxers, també fan moltes altres tasques automàtiques:

- **Minificació:** eliminen espais i comentaris per reduir la mida dels fitxers.
- **Transpilació:** converteixen codi modern (ES6, ES7, etc.) a versions compatibles amb navegadors antics.
- **Gestió de dependències:** permeten importar llibreries de `node_modules` amb `import`.
- **Hot Reload:** actualitzen automàticament la pàgina quan guardem canvis en el codi.
- **Optimització per producció:** creen fitxers finals més petits, amb nom únic i versions “catxejables”.

3.2.5 Cas pràctic de projecte amb Vite

Per acabar d’entendre bé els punts anteriors cal posar-ho en pràctica, és a dir, crear un projecte des de zero amb un empaquetador i veure la seva utilitat.

Imaginem un **cas pràctic**: volem crear una **calculadora modular amb Vite**.

Suposem la següent **cronologia procedimental**:

1. Crearem un nou projecte calculadora-modular amb Vite.
2. Eliminarem els fitxers que no ens interessin del projecte de mostra i reaprofitarem l’arxiu d’estils `style.css`
3. Crearem tres mòduls:
 - `operacions.js` → que exporta suma, resta, multiplica, divideix.
 - `interficie.js` → que s’encarregarà de llegir els camps d’un formulari i mostrar el resultat. Importarà les operacions de `operacions.js`
 - `main.js` → serà el punt d’entrada que importa `interficie.js` i `style.css`
4. Afegirem un formulari senzill a `index.html` per introduir dos números i triar l’operació.
5. Executarem `npm run dev` i comprovarem el funcionament.
6. Finalment, executa `npm run build` i examinarem com Vite empaqueta tot el projecte.

Desenvoluparem el projecte seguint els punts de la cronologia:

1. Crear nou projecte:

```
1 npm create vite@latest calculadora-modular
```

4. Validació de formularis amb llibreries

En aquesta lliçó, explorarem com millorar la validació de formularis mitjançant l'ús de llibreries externes. Després d'una introducció a la creació d'una llibreria personalitzada per a validacions, aprendrem a utilitzar solucions més avançades com **JustValidate**, una llibreria lleugera i fàcil d'integrar. Aquesta lliçó inclou tant exemples pràctics com la creació d'un formulari complet amb validacions automàtiques, així com el treball amb eines modernes com Vite per millorar l'estructura i la gestió del projecte.

4.1 Programació d'una llibreria per validar formularis

Una **llibreria** és un conjunt de **funcions o mòduls reutilitzables** que resolen un problema concret.

En el cas dels formularis, una llibreria pot ajudar-nos a:

- Centralitzar totes les funcions de validació.
- Reduir codi repetit.
- Mantenir el codi més net i estructurat.
- Reutilitzar les mateixes regles en diferents pàgines o projectes.

A la lliçó 2 vam validar els formularis manualment, camp per camp. Ara crearem una llibreria que ho faci automàticament, passant-li només les regles i els camps.

4.1.1 Crear una llibreria bàsica de validació

Una llibreria no és més que un fitxer `.js` que conté funcions reutilitzables.

Comencem amb una estructura senzilla que encapsulem dins el fitxer `validacions.js`:

```
1 export function campObligatori(valor) {  
2   return valor.trim() !== "";  
3 }  
4  
5 export function longitudMinima(valor, mida) {  
6   return valor.trim().length >= mida;  
7 }  
8  
9 export function correuValid(valor) {  
10   const patron = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
11   return patron.test(valor);  
12 }
```

Aquestes funcions **retornen true o false** segons si el valor és correcte. Són senzilles però fàcilment ampliables.

Aquest fitxer l'hem creat pensant que serà utilitzat en un altre fitxer .js i és per això que hem exportat la seva funcionalitat, gràcies al fet que ja coneixem el treball amb mòduls.

4.1.2 Utilitzar la llibreria

Farem servir la llibreria validacions.js en el fitxer d'entrada de la nostra aplicació main.js:

```
1 import { campObligatori, longitudMinima, correuValid } from './
  validacions.js';
2
3 function validarFormulari() {
4   const nom = document.getElementById("nom").value;
5   const email = document.getElementById("email").value;
6   const missatge = document.getElementById("missatge");
7
8   if (!campObligatori(nom)) {
9     missatge.textContent = "El camp 'nom' és obligatori.";
10    return false;
11  }
12
13  if (!longitudMinima(nom, 3)) {
14    missatge.textContent = "El nom ha de tenir almenys 3 carà
      cters.";
15    return false;
16  }
17
18  if (!correuValid(email)) {
19    missatge.textContent = "El correu no té un format vàlid.";
20    return false;
21  }
22
23  missatge.textContent = "Formulari correcte!";
24  return true;
25 }
```

4.1.3 Connectar la lògica amb la vista HTML

script index.html:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Validació amb llibreria pròpia</title>
6 </head>
7 <body>
8
```

```
9 <h3>Formulari de contacte</h3>
10 <form id="formulari" onsubmit="return validarFormulari();">
11   <label for="nom">Nom:</label>
12   <input type="text" id="nom">
13
14   <label for="email">Correu electrònic:</label>
15   <input type="email" id="email">
16
17   <button type="submit">Enviar</button>
18   <p id="missatge"></p>
19 </form>
20
21 <script type="module" src="main.js"></script>
22 </body>
23 </html>
```

Fixem-nos que:

- Estem treballant amb mòduls ES6.
- El formulari no s'enviarà mai perquè dins de main.js aturem el comportament per defecte amb `event.preventDefault();`. Si volguéssim enviar el formulari després de les validacions senzillament hauríem de canviar la línia `missatge.textContent = "Formulari correcte!"`; per `document.getElementById("formulari").submit();`.

Ja podem testejar el nostre formulari, i com veiem en la figura 5.1 i la figura 5.2 tenim la funcionalitat ben implementada.

També vam treballar amb mòduls ES6 a la lliçó 3.

FIGURA 4.1. Exemple del codi de validació del formulari amb llibreries en el cas que falti algun camp

The screenshot shows a web browser window with the title "Validació amb llibreria pròpia". The address bar shows the URL "127.0.0.1:5500/index.html". The page content includes a heading "Formulari de contacte". Below the heading, there are two input fields: "Nom:" with the value "Joan" and "Correu electrònic:". The "Enviar" button is visible. Below the inputs, a message states "El correu no té un format vàlid." (The email does not have a valid format).

FIGURA 4.2. Exemple del codi de validació del formulari amb llibreries en el cas que la validació sigui correcta



4.1.4 Afegir missatges personalitzats i estil visual

Podem millorar tot el codi anterior per tal que els missatges d'error ens sortissin just sota l'element equivocat amb un estil visual adequat per millorar l'experiència de l'usuari.

Al fitxer `index.html` definirem uns estils i crearem unes etiquetes ocultes amb els **missatges d'error**:

```

1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Formulari amb validació (mòduls)</title>
6     <meta name="viewport" content="width=device-width, initial-
7       scale=1" />
8     <style>
9       form { max-width: 420px; margin: 2rem auto; font-family:
10         system-ui, sans-serif; }
11       label { display: block; margin-top: 1rem; font-weight:
12         600; }
13       input { width: 100%; padding: .6rem; margin-top: .35rem;
14         border: 1px solid #bbb; border-radius: .5rem; }
15       button { margin-top: 1rem; padding: .7rem 1rem; border: 0;
16         border-radius: .6rem; cursor: pointer; }
17       .error { display: block; margin-top: .25rem; }
18     </style>
19   </head>
20   <body>
21     <h3 style="text-align:center">Formulari de contacte</h3>
22
23     <form id="formulari">
24       <div>
25         <label for="nom">Nom</label>
26         <input type="text" id="nom" name="nom" />
27         <span
28           class="error"

```

```
24         style="color: red; font-size: 0.9rem; display: none;"
25         >El nom és obligatori.</span>
26     >
27 </div>
28
29 <div>
30     <label for="email">Correu electrònic</label>
31     <input type="email" id="email" name="email" />
32     <span
33         class="error"
34         style="color: red; font-size: 0.9rem; display: none;"
35         >El correu electrònic és obligatori.</span>
36     >
37 </div>
38
39 <button type="submit">Enviar</button>
40 <p id="missatge"></p>
41 </form>
42
43 <script type="module" src="./main.js"></script>
44 </body>
45 </html>
```

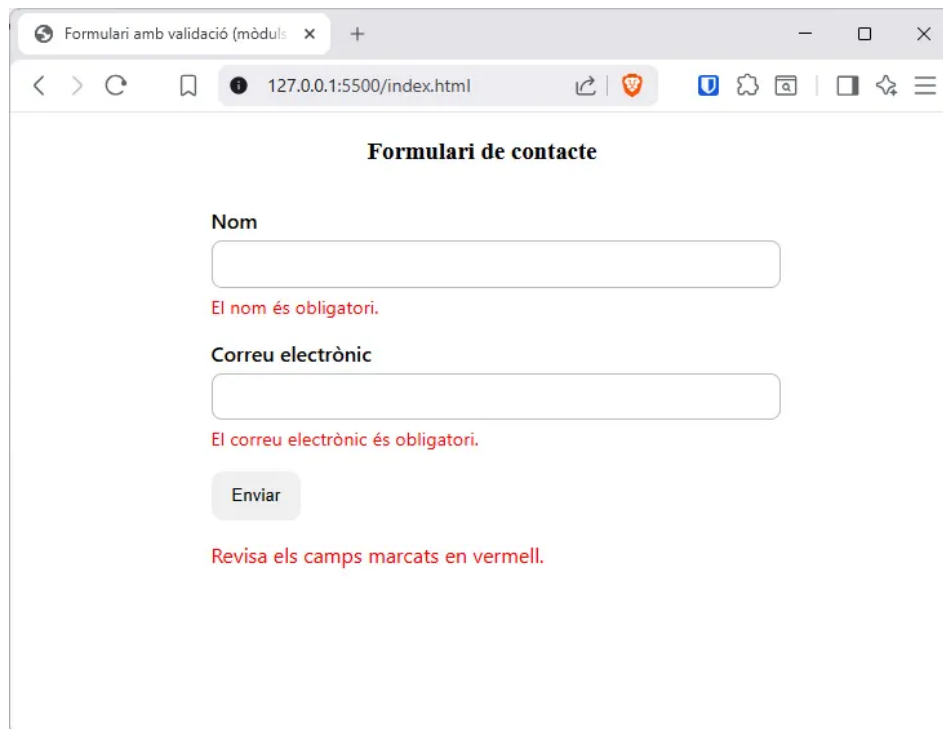
I ampliarem l'arxiu main.js afegint els canvis:

```
1  import { campObligatori, longitudMinima, correuValid } from './
   validacions.js';
2
3  function mostraError(input) {
4      let span = input.parentElement.querySelector('.error');
5      span.style.display = 'block';
6  }
7
8  function netejaError(input) {
9      const span = input.parentElement.querySelector('.error');
10     span.style.display = 'none';
11 }
12
13 function validaCampNom(input) {
14     const valor = input.value;
15     if (!campObligatori(valor)) {
16         mostraError(input);
17         return false;
18     }
19     if (!longitudMinima(valor, 2)) {
20         mostraError(input);
21         return false;
22     }
23     netejaError(input);
24     return true;
25 }
26
27 function validaCampEmail(input) {
28     const valor = input.value;
29     if (!campObligatori(valor)) {
30         mostraError(input, 'El correu és obligatori.');
```

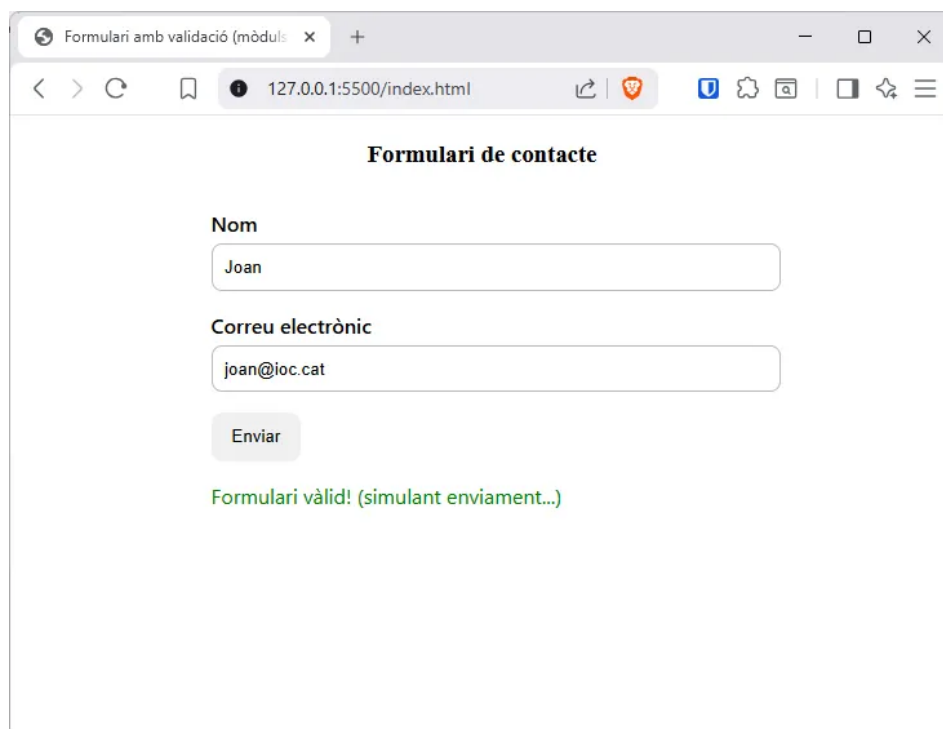


```
36   }
37   netejaError(input);
38   return true;
39 }
40
41 const form = document.getElementById('formulari');
42 const nom = document.getElementById('nom');
43 const email = document.getElementById('email');
44 const missatge = document.getElementById('missatge');
45
46 // Validació en esdeveniments de sortida/canvi per millorar UX
47 // Quan l'usuari surt del camp, el validem
48 nom.addEventListener('blur', () => validaCampNom(nom));
49 email.addEventListener('blur', () => validaCampEmail(email));
50
51 form.addEventListener('submit', (e) => {
52   e.preventDefault(); // evita l'enviament automàtic
53
54   const nomOK = validaCampNom(nom);
55   const emailOK = validaCampEmail(email);
56
57   if (nomOK && emailOK) {
58     missatge.textContent = 'Formulari vàlid! (simulant
59       enviament...)';
60     missatge.style.color = 'green';
61
62     nom.netejaError();
63     email.netejaError();
64   } else {
65     missatge.textContent = 'Revisa els camps marcats en
66       vermell.';
67     missatge.style.color = 'red';
68   }
69 });
```

La nostra llibreria `validacions.js` com que no té cap relació amb la vista no pateix cap canvi respecte a la versió del punt anterior. El comportament del formulari després d'afegir la capa d'estils el podem veure en la figura 5.3 i la figura 5.4.

FIGURA 4.3. Formulari amb camps obligatoris. Missatge d'error

The screenshot shows a web browser window with the title "Formulari amb validació (mòdul: x)". The address bar shows "127.0.0.1:5500/index.html". The page content is titled "Formulari de contacte". It contains two input fields: "Nom" and "Correu electrònic". Below the "Nom" field, there is a red error message: "El nom és obligatori." Below the "Correu electrònic" field, there is a red error message: "El correu electrònic és obligatori." Below these fields is a button labeled "Enviar". At the bottom of the form, there is a red message: "Revisa els camps marcats en vermell."

FIGURA 4.4. Es compleixen les regles de validació

The screenshot shows the same web browser window as Figure 4.3. The "Nom" field now contains the text "Joan" and the "Correu electrònic" field contains the text "joan@ioc.cat". The "Enviar" button is still present. Below the button, there is a green message: "Formulari vàlid! (simulant enviament...)"

Fixem-nos que per aconseguir aquest estil visual hem hagut de manipular manualment el DOM, mostrant i ocultant etiquetes, que és quelcom que estudiarem amb més calma més endavant.

Aquesta estructura és **molt flexible i pot créixer** fins a convertir-se en una llibreria completa de validació per reutilitzar-la en altres projectes.

4.2 Llibreries per validar formularis

Quan el projecte creix, escriure totes les validacions manualment pot resultar feixuc.

Per això, hi ha **llibreries ja preparades** que ens ajuden a validar formularis de manera més ràpida i consistent. A la taula 4.1 podeu veure algunes de les més conegudes:

TAULA 4.1. Llibreries de validació de formularis

Llibreria	Característiques principals
JustValidate	Lleugera, sense dependències, molt fàcil d'integrar en projectes moderns.
Parsley.js	Proporciona validació declarativa amb atributs HTML i missatges personalitzables.
Validator.js	Centrada en validar valors (emails, telèfons, dates, números, etc.) dins del JS, no directament al DOM.
Yup	Sovint usada amb React i <i>frameworks</i> moderns per validar objectes complexos.
Formik + Yup	Combinació molt utilitzada per gestionar i validar formularis en React.

4.2.1 Exemple pràctic amb JustValidate

Escollim **JustValidate** com a llibreria de validació de formularis, i amb ella farem un exemple pràctic. El procés serà el següent:

1. Afegir la llibreria
2. Crear el formulari
3. Configurar la validació
4. Resultat final

Afegir la llibreria

Podem importar **JustValidate** des de CDN al nostre HTML:

```
1 <script src="https://cdn.jsdelivr.net/npm/just-validate@latest/dist/just-validate.production.min.js"></script>
```

Un **CDN** (*Content Delivery Network*) és una xarxa de servidors distribuïts geogràficament que permet que els fitxers es carreguin més ràpidament des del servidor més proper a la ubicació de l'usuari.

Crear el formulari

```
1 <form id="formulariContacte">
2   <div class="form-group">
3     <label for="nom">Nom complet:</label>
4     <input type="text" id="nom" name="nom" />
5   </div>
6   <div class="form-group">
7     <label for="email">Correu electrònic:</label>
8     <input type="email" id="email" name="email" />
9   </div>
10  <div class="form-group">
11    <label for="edat">Edat:</label>
12    <input type="number" id="edat" name="edat" />
13  </div>
14  <button type="submit">Enviar</button>
15 </form>
16
17 <p id="missatge"></p>
```

Configurar la validació

Quan usem una llibreria com **JustValidate**, no cal que nosaltres escrivim manualment totes les funcions per comprovar si un camp està buit, si té prou caràcters o si un correu té el format correcte.

La llibreria ja ens ofereix **regles predefinides** que podem aplicar fàcilment a cada camp del formulari.

Aquí tenim el codi:

```
1 <script>
2   const validator = new JustValidate('#formulariContacte');
3
4   validator
5     .addField('#nom', [
6       { rule: 'required', errorMessage: 'El nom és obligatori' },
7       { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 caràcters' },
8     ])
9     .addField('#email', [
10      { rule: 'required', errorMessage: 'El correu és obligatori' },
11      { rule: 'email', errorMessage: 'El correu no és vàlid' },
12    ])
13     .addField('#edat', [
14      { rule: 'required', errorMessage: 'L'edat és obligatòria' },
15    ])
```

```
15     { rule: 'number', errorMessage: 'Només es permeten números  
16         ' },  
17     { rule: 'minNumber', value: 18, errorMessage: 'Has de ser  
18         major d'edat' },  
19     ])  
20     .onSuccess(() => {  
21         document.getElementById("missatge").textContent = "  
    Formulari correcte!";  
    });  
</script>
```

Abans d'escriure el codi anterior, ens hem hagut de preguntar:

1. Quin formulari volem validar?

Volem validar al formulari amb *id* `formulariContacte`. Això ho aconseguim creat l'objecte:

```
1 const validator = new JustValidate('#formulariContacte');
```

2. Quins camps del formulari volem comprovar?

- Nom
- Email
- Edat

3. Quines regles s'han d'aplicar a cada camp?

- Pel camp **nom** volem que:
 - Existeixi.
 - Tingui una longitud mínima de 3 caràcters.
- Pel camp **email** volem que:
 - Existeixi.
 - Tingui el format que **JustValidate** consideri que ha de tenir una adreça de correu electrònic.
- Pel camp **edat** volem que:
 - Existeixi.
 - Sigui de tipus numèric.
 - El valor sigui més gran que 18.

4. Quins missatges d'error s'han de mostrar si no es compleix una regla?

Cada regla té l'atribut `errorMessage` on definim el missatge en cas que no es compleixi la regla.

Aquesta llibreria detecta automàticament els errors i mostra els missatges al costat dels camps sense necessitat d'escriure gaire codi.

Resultat final

Si posem tot el codi anterior en un sol fitxer .html i afegim una mica d'estil tenim el següent codi:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7     <!-- Importar JustValidate des de CDN -->
8     <script src="https://cdn.jsdelivr.net/npm/just-
      validate@latest/dist/just-validate.production.min.js"></
      script>
9   <style>
10    /* Estils bàsics per al formulari i els missatges d'error
      */
11    body {
12      font-family: Arial, sans-serif;
13      margin: 20px;
14    }
15    form {
16      max-width: 400px;
17      padding: 20px;
18      border: 1px solid #ccc;
19      border-radius: 8px;
20    }
21    .form-group {
22      margin-bottom: 15px;
23    }
24    label {
25      display: block;
26      margin-bottom: 5px;
27      font-weight: bold;
28    }
29    input[type="text"],
30    input[type="number"],
31    input[type="email"] {
32      width: 100%;
33      padding: 10px;
34      border: 1px solid #ccc;
35      border-radius: 4px;
36      box-sizing: border-box; /* Important per incloure
      padding al width */
37    }
38    button {
39      background-color: #4CAF50;
40      color: white;
41      padding: 10px 15px;
42      border: none;
43      border-radius: 4px;
44      cursor: pointer;
45    }
46    /* Estil per als missatges d'error de JustValidate */
47    .js-validate-error-label {
48      color: red;
49      font-size: 0.9em;
50      margin-top: 5px;
51    }
52  </style>
```

```
53 </head>
54 <body>
55   <div class="container">
56     <h3>Formulari de Contacte</h3>
57     <form id="formulariContacte">
58       <div class="form-group">
59         <label for="nom">Nom complet:</label>
60         <input type="text" id="nom" name="nom" />
61       </div>
62       <div class="form-group">
63         <label for="email">Correu electrònic:</label>
64         <input type="email" id="email" name="email" />
65       </div>
66       <div class="form-group">
67         <label for="edat">Edat:</label>
68         <input type="number" id="edat" name="edat" />
69       </div>
70       <button type="submit">Enviar</button>
71     </form>
72     <p id="missatge"></p>
73   </div>
74
75   <script>
76     // Inicialitzar JustValidate
77     const validator = new JustValidate('#formulariContacte', {
78       // Configuració opcional per localitzar o canviar l'
79       // estil
80       errorFieldCssClass: 'is-invalid',
81       errorLabelCssClass: 'js-validate-error-label',
82     });
83
84     // Afegir regles de validació per als camps
85     validator
86       .addField('#nom', [
87         { rule: 'required', errorMessage: 'El nom és
88           obligatori' },
89         { rule: 'minLength', value: 3, errorMessage: 'Mínim 3
90           caràcters' },
91       ])
92       .addField('#email', [
93         { rule: 'required', errorMessage: 'El correu és
94           obligatori' },
95         { rule: 'email', errorMessage: 'El correu no és vàlid'
96           },
97       ])
98       .addField('#edat', [
99         { rule: 'required', errorMessage: "L'edat és obligatò
100           ria" },
101         { rule: 'number', errorMessage: 'Només es permeten nú
102           meros' },
103         { rule: 'minNumber', value: 18, errorMessage: "Has de
104           ser major d'edat" },
105       ])
106       .onSuccess(() => {
107         // Mostrar missatge d'èxit quan totes les validacions
108         // són correctes
109         document.getElementById("missatge").textContent = "
110           Formulari correcte!";
111         document.getElementById("missatge").className = "
112           success-message";
113       });
114   </script>
115 </body>
```

```
105 </html>
```

Ens hem de fixar que hem afegit algun detall més al constructor de l'objecte validator per tal que ens mostri els errors sota el camp input que conté l'error:

```
1 const validator = new JustValidate("#formulariContacte", {  
2   // Configuració opcional per localitzar o canviar l'estil  
3   errorFieldCssClass: "is-invalid",  
4   errorLabelCssClass: "js-validate-error-label",  
5 });
```

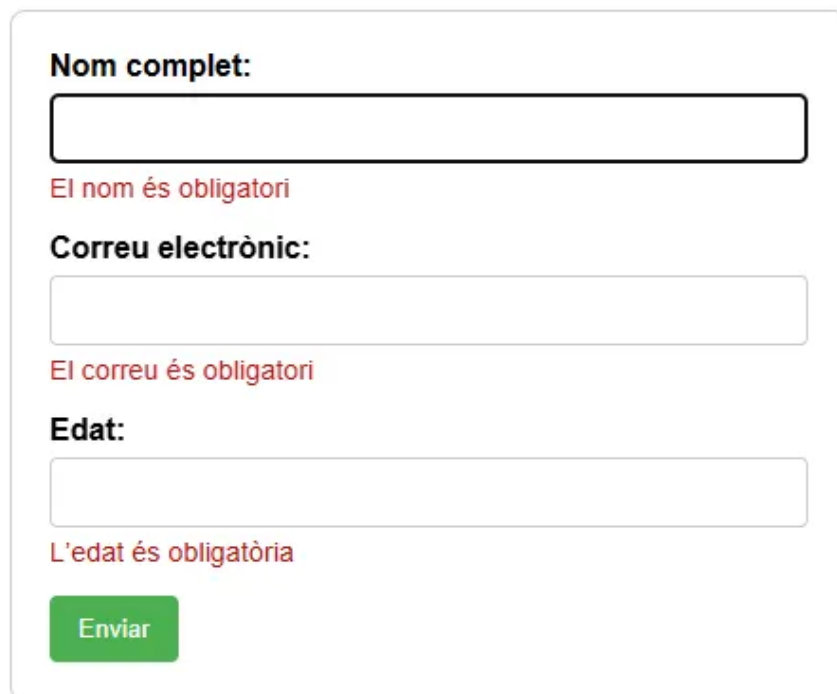
I amb el codi anterior aconseguim un formulari semblant al de la figura 5.5.

FIGURA 4.5. Formulari de contacte amb camps obligatoris

Llibreria JustValidate

Podem trobar la documentació de la llibreria JustValidate a just-validate.dev.

Formulari de Contacte



Formulari de Contacte

Nom complet:

El nom és obligatori

Correu electrònic:

El correu és obligatori

Edat:

L'edat és obligatòria

Envíar

4.2.2 Treball modern amb JustValidate

Ara reescriurem l'exemple del punt anterior, però des d'una aproximació més moderna, aprofitant l'aprenentatge d'ús d'empaquetadors de la lliçó anterior. El procés serà el següent:

1. Crear un nou projecte amb Vite
2. Instal·lació de JustValidate
3. Creació del formulari i les regles de validació

4. Execució del projecte

Crear un nou projecte amb Vite

Crearem un nou projecte amb **Vite** i utilitzem la plantilla **vanilla**:

```
1 npm create vite@latest projecte-justvalidate
```

Si no ens ho fa sol, llençarem `npm install` per tenir les dependències instal·lades.

Instal·lació de JustValidate

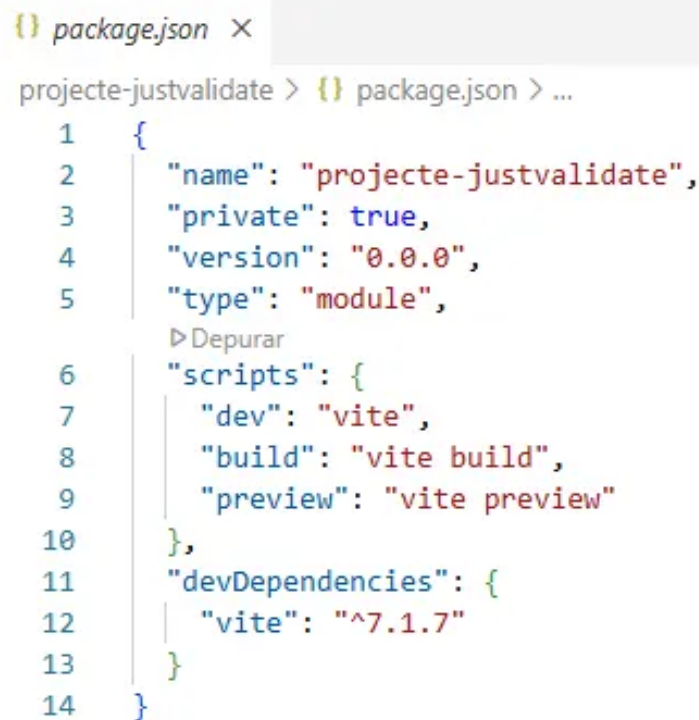
Un cop tinguem el projecte de Vite creat i les dependències instal·lades, hem d'afegir JustValidate al projecte. Dins de la ruta del projecte farem:

```
1 npm install just-validate
```

Al fer l'acció anterior, el que estem fent, a part d'instal·lar l'eina que necessitem, és escriure en el fitxer on hi ha les referències del projecte, `package.json`, que aquest projecte requereix l'eina `just-validate`.

Fixem-nos com teníem l'arxiu `package.json` abans d'instal·lar `just-validate` (figura 5.6):

FIGURA 4.6. fitxer `package.json` abans d'instal·lar "just-validate".



```
{ package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   }
14 }
```

I tot seguit el mateix arxiu just després d'instal·lar `just-validate` (figura 5.7):

FIGURA 4.7. fitxer package.json després d'instal·lar "just-validate".

```
{} package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   },
14   "dependencies": {
15     "just-validate": "^4.3.0"
16   }
17 }
18
```

L'arxiu **package.json** és un fitxer essencial en projectes de Node.js que conté informació sobre el projecte, com el nom, la versió i les dependències que necessita per funcionar. Aquestes dependències són altres paquets o llibreries que el projecte utilitza, i estan especificades en el camp `dependencies` o `devDependencies`. Quan executeu el comandament `npm install`, Node Package Manager (NPM) llegeix el `package.json`, descarrega i instal·la totes les dependències esmentades a les versions especificades, assegurant-se que el projecte tingui totes les llibreries necessàries per funcionar correctament.

Creació del formulari i les regles de validació

El formulari de validació, `index.html` per exemple, serà exactament el mateix que en l'exemple anterior. Ara, però, caldrà agafar el punt d'entrada de la lògica que serà el fitxer `main.js`.

Fixem-nos que ara el fitxer `main.js` l'afegim dins de la carpeta `src/` seguint l'estructura que Vite crea per defecte:

```
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7   </head>
8   <body>
9     <div class="container">
10      <h3>Formulari de Contacte</h3>
11      <form id="formulariContacte">
12        <div class="form-group">
13          <label for="nom">Nom complet:</label>
14          <input type="text" id="nom" name="nom" />
15        </div>
16        <div class="form-group">
17          <label for="email">Correu electrònic:</label>
18          <input type="email" id="email" name="email" />
19        </div>
20        <div class="form-group">
21          <label for="edat">Edat:</label>
22          <input type="number" id="edat" name="edat" />
23        </div>
24        <button type="submit">Enviar</button>
25      </form>
26      <p id="missatge"></p>
27    </div>
28    <script type="module" src="src/main.js"></script>
29  </body>
30 </html>
```

Ara escrivim les regles de validació a main.js:

```
1 import JustValidate from 'just-validate';
2 import "./style.css";
3
4 const validator = new JustValidate('#formulariContacte');
5
6 validator
7   .addField('#nom', [
8     { rule: 'required', errorMessage: 'El nom és obligatori' },
9     { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 carà
      cters' },
10  ])
11   .addField('#email', [
12     { rule: 'required', errorMessage: 'El correu és obligatori'
      },
13     { rule: 'email', errorMessage: 'El correu no és vàlid' },
14  ])
15   .addField('#edat', [
16     { rule: 'required', errorMessage: "L'edat és obligatòria" },
17     { rule: 'number', errorMessage: 'Només es permeten números'
      },
18     { rule: 'minNumber', value: 18, errorMessage: "Has de ser
      major d'edat" },
19  ])
20   .onSuccess(() => {
21     document.getElementById('missatge').textContent = 'Formulari
      correcte!';
22   });
```

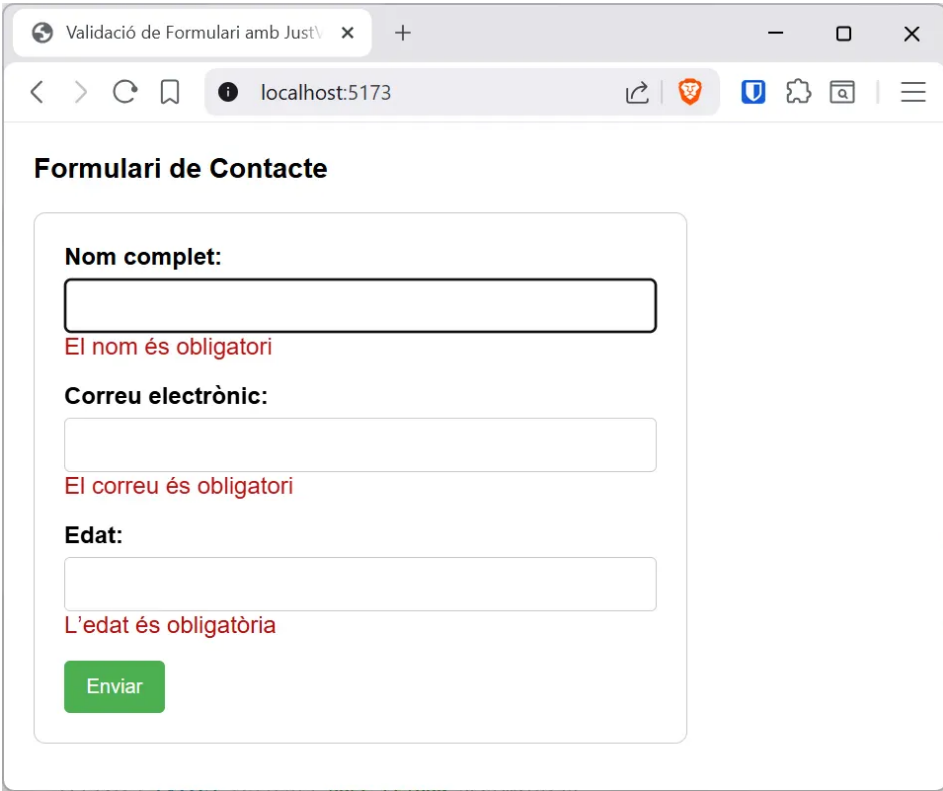
I afegirem els estils a l'arxiu `style.css`:

```
1  body {
2    font-family: Arial, sans-serif;
3    margin: 20px;
4  }
5
6  form {
7    max-width: 400px;
8    padding: 20px;
9    border: 1px solid #ccc;
10   border-radius: 8px;
11 }
12
13 .form-group {
14   margin-bottom: 15px;
15 }
16
17 label {
18   display: block;
19   margin-bottom: 5px;
20   font-weight: bold;
21 }
22
23 input[type='text'],
24 input[type='number'],
25 input[type='email'] {
26   width: 100%;
27   padding: 10px;
28   border: 1px solid #ccc;
29   border-radius: 4px;
30   box-sizing: border-box;
31 }
32
33 button {
34   background-color: #4CAF50;
35   color: white;
36   padding: 10px 15px;
37   border: none;
38   border-radius: 4px;
39   cursor: pointer;
40 }
41
42 .js-validate-error-label {
43   color: red;
44   font-size: 0.9em;
45   margin-top: 5px;
46 }
47
48 .is-invalid {
49   border-color: red;
50 }
```

Execució del projecte

Un cop tinguem tots els fitxers configurats, executarem el servidor de desenvolupament amb **Vite** (figura 4.8):

```
1  npm run dev
```

FIGURA 4.8. Execució del formulari amb validació de camps

The screenshot shows a web browser window with the title "Validació de Formulari amb Just". The address bar displays "localhost:5173". The page content is titled "Formulari de Contacte" and contains three input fields, each with a red validation message below it:

- Nom complet:** The input field is empty, and the message "El nom és obligatori" is displayed in red.
- Correu electrònic:** The input field is empty, and the message "El correu és obligatori" is displayed in red.
- Edat:** The input field is empty, and the message "L'edat és obligatòria" is displayed in red.

At the bottom of the form is a green button labeled "Enviar".